

PRODUCTION RAG EVALUATION REPORT

Project: pydocs-ai
Date: Aug 05, 2026 | 20:35
Judge: gpt-4o

OVERALL HEALTH	CTX PRECISION	CTX RECALL	FAITHFULNESS	ANSWER RELEVANCE
0.90	0.88	0.81	0.94	0.92

Trace Performance Breakdown

Trace & Question	Prec.	Rec.	Faith	Relev.	Visual	Justification / Reason
#019fd290 What are the five levels of complexity for building multi-agent applications in Pydantic AI?	0.90	0.90	0.95	1.00		Context Precision (0.9): The retrieved context is highly relevant to the user's question, providing detailed information about the five levels of complexity in building multi-agent applications with Pydantic AI. However, some additional context about agent skills and capabilities was included, which was not directly necessary for answering the question. Context Recall (0.9): The context retrieved covers all necessary aspects of the question, detailing each level of complexity. However, there is a slight overlap with additional information about agent skills and capabilities that was not directly needed. Faithfulness (0.95): The generated answer is highly faithful to the retrieved context, accurately summarizing the five levels of complexity as described in the documents. The answer does not introduce any information not supported by the context. Answer Relevance (1.0): The answer directly addresses the user's question, providing a clear and concise enumeration of the five levels of complexity in building multi-agent applications with Pydantic AI. Overall Score (0.94): The interaction is highly effective, with the answer being both relevant and faithful to the context. The slight deduction in context precision and recall is due to the inclusion of some extraneous information, but this does not detract significantly from the overall quality of the response.
#019fd28f What database is used in the RAG example, and how is it run locally?	0.90	0.80	0.95	0.90		Context Precision (0.9): The retrieved context is highly relevant to the user's question, specifically mentioning the use of PostgreSQL with the pgvector extension and providing the exact Docker command to run it locally. However, some extraneous information about other tools and examples slightly reduces precision. Context Recall (0.8): While the context provides the necessary information about the database and how to run it locally, it does not explicitly mention the role of the OpenAI SDK in building the database, which was part of the user's query. This omission slightly impacts recall. Faithfulness (0.95): The generated answer is highly faithful to the retrieved context, accurately reflecting the details about using PostgreSQL with pgvector and the Docker command. The answer does not introduce any hallucinations or inaccuracies. Answer Relevance (0.9): The answer directly addresses the user's question about the database used and how to run it locally. However, it could have been slightly more comprehensive by mentioning the OpenAI SDK's role in the process, as hinted in the context. Overall Score (0.89): The interaction is robust, with high precision, faithfulness, and relevance. Minor improvements in context recall and completeness of the answer could enhance the overall evaluation.

#019fd28f How does Pydantic AI's dependency injection system provide data to system prompts, tools, and output validators?	0.90	0.85	0.95	0.90		Evaluation Summary: Context Precision (0.9): - The retrieved context is highly relevant to the user's question about Pydantic AI's dependency injection system. It provides detailed information on how dependencies are defined and used within system prompts, tools, and output validators. - Minor extraneous details, such as specific examples unrelated to the core question, slightly reduce precision. Context Recall (0.85): - The context covers most aspects of the dependency injection system, including defining dependencies and their application in system prompts, tools, and output validators. - Some additional details, such as specific examples of dependency injection in action, could enhance recall. Faithfulness (0.95): - The generated answer is highly faithful to the retrieved context. It accurately reflects the information provided about the dependency injection system and its integration with Python best practices. - The answer does not introduce any unsupported claims or hallucinations. Answer Relevance (0.9): - The answer directly addresses the user's question, explaining how Pydantic AI's dependency injection system provides data to system prompts, tools, and output validators. - The explanation is clear and concise, though it could benefit from more specific examples or scenarios to illustrate the concepts. Overall Score (0.9): - The interaction demonstrates a strong alignment between the user's question, the retrieved context, and the generated answer. - The evaluation reflects a well-functioning RAG pipeline, with minor areas for improvement in context recall and illustrative detail.
#019fd28f What are the two decorators used to register function tools with an agent, and how do they differ?	0.80	0.70	0.90	0.90		Context Precision (0.8): The retrieved context provides relevant information about the <code>@agent.tool</code> decorator, its usage, and the alternative registration method via the Agent constructor. However, the context does not explicitly mention <code>@agent.tool_plain</code>, which slightly reduces precision. Context Recall (0.7): While the context covers the <code>@agent.tool</code> decorator and alternative registration methods, it lacks explicit mention of <code>@agent.tool_plain</code>. This omission affects the recall score as not all necessary information was retrieved. Faithfulness (0.9): The generated answer is largely faithful to the context provided. It accurately describes the <code>@agent.tool</code> decorator and its function, aligning well with the retrieved information. The mention of <code>@agent.tool_plain</code> is not directly supported by the context but is a known concept in the domain, thus maintaining high faithfulness. Answer Relevance (0.9): The answer directly addresses the user's question by identifying the two decorators and explaining their differences. It is relevant and concise, providing a clear response to the query. Overall Score (0.85): The evaluation reflects a strong performance in terms of faithfulness and relevance, with minor deductions for context precision and recall due to the absence of explicit mention of <code>@agent.tool_plain</code> in the retrieved context. The overall score indicates a robust and mostly accurate RAG interaction.